

코딩테스트 체크리스트 — 모범답안

1단계: 메서드 암기

Java — ArrayList (13)

1. ArrayList 끝에 요소를 추가하는 메서드는? `add(e)` — $O(1)$
2. ArrayList의 특정 인덱스에 요소를 삽입하는 코드를 써라 `list.add(index, e)` — $O(n)$, 뒤의 요소를 한 칸씩 밀어야 하므로
3. ArrayList에서 인덱스 2의 요소를 제거하는 코드와 값 2를 제거하는 코드를 각각 써라

```
java  
  
list.remove(2);           // 인덱스 2 제거  
list.remove(Integer.valueOf(2)); // 값 2 제거
```

Java는 int와 Integer 중 int를 우선 매칭하므로, 값으로 제거하려면 반드시 `Integer.valueOf()` 사용

4. ArrayList에서 인덱스 접근(읽기)하는 메서드는? 시간복잡도는? `get(index)` — $O(1)$. 배열 기반이므로 주소 + offset 계산으로 바로 접근
5. ArrayList에서 특정 인덱스의 값을 변경하는 메서드는? `set(index, e)` — $O(1)$
6. ArrayList의 크기를 반환하는 메서드는? 왜 $O(1)$ 인가? `size()` — 내부에 size 필드를 두고 add/remove 시마다 갱신해두기 때문. 매번 세지 않음
7. ArrayList에서 부분 리스트를 추출하는 메서드는? 반환값은 복사본인가 뷰인가? `subList(from, to)` — 뷰(view). 복사하지 않고 원본 리스트의 참조 + from/to 인덱스만 저장. 뷰를 수정하면 원본도 변경됨. 독립 복사본이 필요하다면 `new ArrayList<>(list.subList(from, to))`
- ⚠ 실전 주의: 뷰를 만든 상태에서 원본 리스트의 구조적 변경(add/remove)이 일어나면, 이후 뷰를 사용할 때 `ConcurrentModificationException` 발생. `set()`으로 값만 바꾸는 건 괜찮지만, 요소 추가/삭제는 안 됨
8. ArrayList를 정렬하는 메서드는? 내부적으로 어떤 알고리즘을 쓰는가? `list.sort(comparator)` — 내부적으로 TimSort (Merge Sort + Insertion Sort 하이브리드). $O(n \log n)$
9. ArrayList에서 특정 값의 첫 번째 인덱스를 찾는 메서드는? 없으면 뭘 반환하는가? `indexOf(o)` — 없으면 -1 반환. $O(n)$
10. ArrayList에서 값이 존재하는지 확인하는 메서드는? 시간복잡도는? `contains(o)` — $O(n)$. 내부적으로 indexOf를 호출하여 순차 탐색
11. ArrayList가 비어있는지 확인하는 메서드는? `isEmpty()` — $O(1)$. 내부적으로 `size == 0` 비교
12. ArrayList를 스택처럼 쓸 때 push/peek/pop에 해당하는 메서드 조합은?

```
java

list.add(1);           // push
list.get(list.size() - 1); // peek
list.remove(list.size() - 1); // pop
```

13. Java에서 스택이 필요할 때 Stack 클래스 대신 뭘 쓰는가? 왜? `ArrayDeque`를 쓴다. Stack은 Vector를 상속받았고, Vector는 모든 메서드에 synchronized가 걸려있어서 단일 스레드에서도 불필요한 동기화 오버헤드 발생. Java 공식 문서에서도 Deque(ArrayDeque) 사용을 권장

Java — HashMap (15)

14. HashMap에 키-값을 저장하는 메서드는? 키가 이미 있으면 어떻게 되는가? `put(key, value)` — 키가 이미 있으면 값을 덮어씀. 추가와 수정을 모두 처리

15. HashMap에 키가 없을 때만 저장하는 메서드는? `putIfAbsent(key, value)` — 키가 이미 있으면 아무것도 하지 않음

16. HashMap에서 키로 값을 조회하는 메서드는? 키가 없으면 뭘 반환하는가? `get(key)` — 키가 없으면 null 반환

17. HashMap에서 키가 없으면 기본값을 반환하되 원본은 변경하지 않는 메서드는? `getOrDefault(key, default)` — 조회 결과에만 기본값을 반환. 원본 HashMap에는 아무것도 추가하거나 변경하지 않음

18. HashMap의 merge 메서드로 빈도수를 세는 코드를 한 줄로 써라

```
java

map.merge(word, 1, Integer::sum);
// 키가 없으면 1 저장, 있으면 기존값 + 1
```

19. merge에서 사용하는 `::`는 무슨 문법인가? 메서드 참조(method reference). Java 8에서 도입. 람다를 짧게 쓰는 문법으로, merge 전용이 아니라 람다가 들어가는 곳이면 어디서든 사용 가능

```
java

map.merge(word, 1, (oldVal, newVal) -> oldVal + newVal); // 람다
map.merge(word, 1, Integer::sum); // 메서드 참조 (동일)
```

20. HashMap에서 키 존재 여부를 확인하는 메서드는? 시간복잡도는? `containsKey(key)` — $O(1)$. 해시 함수로 버킷 위치를 바로 계산

21. HashMap에서 값 존재 여부를 확인하는 메서드는? 왜 $O(n)$ 인가? `containsValue(value)` — $O(n)$. 값은 해시 함수로 위치를 역추적할 수 없어서 전체를 순회해야 함

22. HashMap의 모든 키를 반환하는 메서드는? 반환 타입은? 왜 Set인가? `keySet()` — Set 반환. 키는 HashMap에서 중복이 불가능하므로 중복을 허용하지 않는 Set이 적절

23. HashMap의 모든 값을 반환하는 메서드는? 반환 타입은? 왜 Collection인가? `values()` — Collection 반환. 값은 중복이 가능하므로 (`{ "a":1, "b":1 }` 처럼) 중복을 허용하는 Collection으로 반환

24. HashMap의 모든 키-값 쌍을 반환하는 메서드는? `entrySet()` — `Set<Map.Entry<K, V>>` 반환

💡 코테 팁: 타입을 다 적기 번거로우면 Java 10+ 환경에서 `for (var entry : map.entrySet())`로 `var` 키워드를 활용하면 타이핑이 빨라짐

25. HashMap에 `set()` 메서드가 없는 이유는? `put(key, value)`이 추가와 수정을 모두 처리하기 때문. 키가 없으면 새로 추가, 키가 이미 있으면 값을 덮어씀

26. `getOrDefault`와 `putIfAbsent`의 차이는?

- `getOrDefault`: 조회만 함. 원본 변경 없음
- `putIfAbsent`: 키가 없으면 실제로 맵에 저장까지 함

27. `merge`와 `getOrDefault+put` 중 어떤 상황에서 어느 것을 쓰는가?

- 단순 누적 (빈도수 세기 등) → `merge`가 간결
- 조건 분기가 복잡한 경우 → `getOrDefault + put`으로 풀어쓰는 게 읽기 편함

28. HashMap의 순서가 필요하면 무엇을 쓰는가? `LinkedHashMap` — HashMap + 키들 사이에 삽입 순서대로 이중 연결 리스트(doubly linked list) 유지

Java — Comparator (6)

29. 람다로 오름차순 정렬하는 Comparator를 써라

```
java
(a, b) -> a - b
```

30. 람다로 내림차순 정렬하는 Comparator를 써라

```
java
(a, b) -> b - a
```

31. `Comparator.comparingInt()`로 문자열을 길이순 정렬하는 코드를 써라

```
java
list.sort(Comparator.comparingInt(String::length));
```

32. 정렬 순서를 뒤집는 메서드는? `reversed()` — 예: `Comparator.comparingInt(String::length).reversed()`

33. 1차 기준 정렬 후 2차 기준 정렬을 추가하는 메서드는? `thenComparing()` — 예: `Comparator.comparingInt(String::length).thenComparing(Comparator.naturalOrder())`

34. Comparable과 Comparator의 차이는? 코테에서 어느 걸 쓰는가?

- Comparable: 객체 내부에 구현 (`compareTo`), "나는 기본적으로 이렇게 정렬된다"
- Comparator: 객체 외부에서 정의 (`compare`), "이 기준으로 정렬해라"
- 코테에서는 Comparator를 쓴다. 문제마다 정렬 기준이 다르므로 외부에서 기준을 주입하는 게 유연

Java — Stream (8)

35. 리스트에서 스트림을 여는 코드는?

```
java  
  
list.stream()
```

36. 스트림에서 조건 필터링하는 메서드는? `filter(Predicate)` — 예: `.filter(n -> n > 5)`

37. 스트림에서 각 요소를 변환하는 메서드는? `map(Function)` — 예: `.map(n -> n * 2)`

38. 스트림에서 중복을 제거하는 메서드는? `distinct()`

39. 스트림 결과를 List로 수집하는 코드를 써라 (Java 8 기준)

```
java  
  
.collect(Collectors.toList())
```

Java 16+에서는 `.toList()`도 가능하지만, 코테에서는 `collect(Collectors.toList())`가 안전

💡 참고: 최근 코테 환경(프로그래머스, 백준 등)은 대부분 Java 17 이상을 지원. 반환된 리스트를 나중에 수정할 일이 없다면 `.toList()`가 더 간결. 단, `.toList()`는 불변 리스트를 반환하므로 이후 add/remove 하면 `UnsupportedOperationException` 발생

40. 스트림에서 모든 요소를 하나로 합치는 메서드는? 합계를 구하는 코드를 써라

```
java  
  
.reduce(0, Integer::sum)
```

첫 번째 인자는 초기값(identity)

41. 스트림의 3가지 특성은?

- 원본 불변: 원본 컬렉션을 수정하지 않고 새로운 결과를 생성
- 일회용: 최종 연산 후 재사용 불가 (다시 `stream()` 호출 필요)
- 지연 평가: 중간 연산은 최종 연산이 호출될 때까지 실행되지 않음

42. 중간 연산과 최종 연산의 차이는?

- 중간 연산: Stream을 반환, 체이닝 가능 (filter, map, sorted, distinct 등)
 - 최종 연산: 결과를 반환, 파이프라인 종료 (collect, forEach, count, reduce 등)
-

JavaScript — Array (9)

43. 배열 끝에 추가/제거하는 메서드는? `push(e)` / `pop()` — 둘 다 $O(1)$
44. 배열 앞에 추가/제거하는 메서드는? 시간복잡도는? `unshift(e)` / `shift()` — 둘 다 $O(n)$. 나머지 요소를 전부 밀거나 당겨야 하므로
45. 특정 위치에서 요소를 제거하면서 동시에 새 요소를 삽입하는 메서드는? `splice(start, count, ...items)` — 원본 변경. $O(n)$
46. 원본을 보존하면서 부분 배열을 추출하는 메서드는? `slice(start, end)` — 원본 보존, 복사본 반환. $O(n)$
47. splice와 slice의 차이를 한 문장으로 말해라 splice는 원본을 변경하고, slice는 원본을 보존한다
48. 배열에서 특정 값이 존재하는지 확인하는 메서드는? 시간복잡도는? `includes(e)` — boolean 반환. $O(n)$. 순차 탐색
49. 배열에서 특정 값의 인덱스를 찾는 메서드는? `indexOf(e)` — 없으면 -1 반환. $O(n)$
50. 배열을 문자열로 합치는 메서드는? `join(separator)` — 예: `[1, 2, 3].join("-")` → `"1-2-3"`
51. 2차원 배열을 올바르게 초기화하는 코드를 써라

```
javascript

// 올바른 방법
const grid = Array.from({ length: 3 }, () => new Array(3).fill(0));

// 잘못된 방법 — 모든 행이 같은 배열을 참조
// const bad = new Array(3).fill(new Array(3).fill(0));
```

JavaScript — Object / Map / Set (8)

52. Object에서 값을 읽고 쓰는 문법은?

```
javascript

obj[key]      // 읽기 (없으면 undefined)
obj[key] = value // 쓰기
```

53. Object의 모든 키/값/키-값 쌍을 배열로 반환하는 메서드를 각각 써라

```
javascript
```

```
Object.keys(obj) // 모든 키 배열
Object.values(obj) // 모든 값 배열
Object.entries(obj) // [키, 값] 쌍의 배열
```

54. Object에서 키를 삭제하는 문법은?

```
javascript

delete obj[key]
```

55. Object에서 키 존재 여부를 확인하는 2가지 방법은? 차이는?

- `obj.hasOwnProperty(key)` — 해당 객체 자체의 속성만 확인
- `key in obj` — 프로토타입 체인까지 포함하여 확인

56. Map에서 키-값을 저장/조회/존재 확인하는 메서드를 각각 써라

```
javascript

map.set(key, value) // 저장
map.get(key)        // 조회 (없으면 undefined)
map.has(key)        // 존재 확인 (boolean)
```

57. Object와 Map 중 어떤 상황에서 Map을 써야 하는가? 키가 문자열 외 타입(숫자, 객체 등)일 때. Object는 키를 문자열로 변환해버리므로 Map 필요. 코테에서는 키가 대부분 문자열이라 Object를 주로 씀

58. Set에서 값을 추가/존재 확인/크기 확인하는 메서드를 각각 써라

```
javascript

set.add(value) // 추가 (이미 있으면 무시)
set.has(value) // 존재 확인 — O(1)
set.size       // 크기 (메서드가 아닌 속성)
```

59. 배열의 중복을 제거한 새 배열을 한 줄로 만드는 코드를 써라

```
javascript

const unique = [...new Set(arr)];
```

JavaScript — sort 콜백 (6)

60. JS에서 숫자 배열을 오름차순 정렬하는 코드를 써라

```
javascript
```

```
arr.sort((a, b) => a - b);
```

61. JS에서 sort()를 콜백 없이 호출하면 어떻게 되는가? 왜 위험한가? 모든 요소를 문자열로 변환 후 사전순 정렬.

[10, 9, 1, 2].sort() → [1, 10, 2, 9]가 되어 숫자 정렬이 깨짐

62. 문자열을 사전순으로 정렬하는 콜백을 써라

```
javascript
```

```
(a, b) => a.localeCompare(b)
```

63. 콜백 반환값이 음수/0/양수일 때 각각 무슨 의미인가?

- 음수 → a가 b보다 앞
- 0 → 순서 유지
- 양수 → b가 a보다 앞

64. sort()는 원본을 변경하는가? 원본을 보존하면서 정렬하려면? 원본을 변경한다 (in-place). 보존하려면 스프레드로 복사:

```
javascript
```

```
const sorted = [...arr].sort((a, b) => a - b);
```

65. 다단계 정렬에서 || 연산자를 사용하는 패턴을 써라

```
javascript
```

```
arr.sort((a, b) => a.score - b.score || a.name.localeCompare(b.name));
```

```
// 1차: 점수 오름차순, 같으면 2차: 이름 사전순
```

```
// 1차 비교가 0이면(같으면) || 뒤의 2차 비교로 넘어감
```

JavaScript — 고차함수 (8)

66. 조건에 맞는 요소만 추출하는 메서드는? filter(callback) — 새 배열 반환

67. 각 요소를 변환하는 메서드는? map(callback) — 새 배열 반환

68. 모든 요소를 하나의 값으로 합치는 메서드는? 초기값을 왜 항상 넣어야 하는가? reduce(callback, initial) — 초기 값을 생략하면 첫 번째 요소가 초기값이 되는데, 빈 배열에서 초기값 없이 호출하면 TypeError 발생

69. 조건에 맞는 첫 번째 요소를 반환하는 메서드는? 없으면 뭘 반환하는가? find(callback) — 없으면 undefined

70. 조건에 맞는 첫 번째 인덱스를 반환하는 메서드는? findIndex(callback) — 없으면 -1

71. "하나라도 만족?"과 "모두 만족?"을 확인하는 메서드를 각각 써라

javascript

```
arr.some(callback) // 하나라도 만족하면 true  
arr.every(callback) // 모두 만족하면 true
```

72. `forEach`의 반환값은? 왜 체이닝이 안 되는가? `undefined`를 반환. 따라서 `.forEach().filter()` 같은 체이닝 불가. 부수 효과(side effect) 전용

73. `map`에서 중괄호를 쓸 때 흔히 하는 실수는? `return`을 빼먹으면 `undefined` 배열이 됨

javascript

```
nums.map(n => { n * 2 }); // [undefined, undefined, ...]  
nums.map(n => { return n * 2 }); // 올바른  
nums.map(n => n * 2); // 한 줄이면 중괄호 생략이 가장 깔끔
```

2단계: 패턴 적용

빈도 세기 / 카운팅 (4)

74. 배열에서 각 요소의 등장 횟수를 세야 한다. Java와 JS 각각 어떤 자료구조와 메서드를 쓸 것인가?

- Java: `HashMap` + `getOrDefault` 또는 `merge`
- JS: `Object` + `(obj[key] || 0) + 1` 패턴

75. Java에서 빈도 세기를 `merge`로 한 줄로 쓰는 코드와 `getOrDefault`+`put`으로 풀어쓰는 코드를 각각 써라

java

```
// merge  
map.merge(word, 1, Integer::sum);  
  
// getOrDefault + put  
map.put(word, map.getOrDefault(word, 0) + 1);
```

76. JS에서 빈도 세기의 `(obj[key] || 0) + 1` 패턴을 설명하라 `obj[key]`가 `undefined`(키가 없을 때)이면 `(|| 0)`에 의해 0이 되고, 거기에 1을 더해서 1이 됨. 키가 있으면 기존 값에 1을 더함. Java의 `getOrDefault(key, 0) + 1`과 동일한 동작

77. 빈도를 센 뒤 가장 많이 나온 요소를 찾으려면 어떻게 하는가?

- Java: `entrySet()`으로 순회하며 최대값 추적
- JS: `Object.entries(count)`로 순회하며 최대값 추적
- 둘 다 변수 두 개(`maxKey`, `maxCount`)를 두고 비교

고유 개수 / 중복 제거 (4)

78. HashMap vs HashSet 중 뭘 쓰는가? 판단 기준은?

- 고유 개수만 필요 → HashSet (또는 JS Set)
- 각 종류별 개수까지 필요 → HashMap (또는 JS Object)
- 판단 기준: 값(빈도)이 필요한가?

79. JS에서 배열의 고유 종류 수를 구하는 가장 간단한 코드를 써라

```
javascript  
  
new Set(arr).size
```

80. 고유 개수만이 아니라 각 종류별 개수까지 필요하면 어떻게 접근하는가? HashMap/Object로 빈도를 세고, `keySet().size()` / `Object.keys(obj).length`로 종류 수, 값으로 각 종류별 개수

81. 포켓몬 문제: N/2마리를 골라 최대 종류 수를 구하는 핵심 아이디어는? `Math.min(종류 수, N/2)` — 종류가 선택 가능 수보다 많으면 N/2가 답, 적으면 종류 수가 답

정렬 (4)

82. 숫자 배열을 이어붙여서 가장 큰 수를 만들어라. 정렬 기준을 어떻게 정의하는가? 두 수를 이어붙인 결과를 비교:

```
java  
  
// Java  
Arrays.sort(nums, (a, b) -> (b + a).compareTo(a + b));  
  
// JS  
nums.sort((a, b) => (b + a) - (a + b));
```

"610" vs "106" → "610"이 더 크므로 6이 10보다 앞

83. 1차 기준은 점수 내림차순, 2차 기준은 이름 오름차순으로 정렬하는 코드를 써라

```
java
```

```
// Java — thenComparing
students.sort((a, b) -> {
    if (a[1] != b[1]) return b[1] - a[1];
    return names[a[0]].compareTo(names[b[0]]);
});

// JS — || 연산자
students.sort((a, b) => b.score - a.score || a.name.localeCompare(b.name));
```

84. 내림차순 정렬 후 i 번째 값 * $(i+1)$ 의 최대값을 구하는 문제(로프 문제)의 접근법은? 내림차순 정렬 → 순회하면서 $\text{ropes}[i] * (i + 1)$ 계산 → 최대값 추적. i 번째까지 사용하면 가장 약한 로프가 기준이 되므로

85. Java에서 `sort()`는 원본을 변경하는가? `Stream`의 `sorted()`는?

- `list.sort()`: 원본 변경 (in-place)
- `stream().sorted()`: 원본 불변, 새로운 스트림 반환

부분 문자열 / 존재 여부 검사 (3)

86. `ArrayList`의 `contains` 대신 `HashSet`을 쓰는 이유는? 시간복잡도 차이는?

- `ArrayList.contains()`: $O(n)$ — 순차 탐색
- `HashSet.contains()`: $O(1)$ — 해시로 바로 조회
- 반복문 안에서 호출하면 전체 $O(n^2)$ vs $O(n)$ 차이

87. 접두사 검사에서 `substring`을 어떻게 활용하는가? 접미사는?

```
java

// 접두사: 앞에서부터 자름
phone.substring(0, i);

// 접미사: 뒤에서부터 자름
phone.substring(phone.length() - i);

// 내부 문자열: 모든 구간
phone.substring(i, j);
```

88. 존재 여부만 확인하면 `HashSet`, 부가 정보가 필요하면 `HashMap` — 이 기준을 설명하라

- 존재 여부만 → `HashSet` / `Set` (값이 필요 없으므로)
- 부가 정보 필요 (예: 전화번호 → 사용자 이름) → `HashMap` / `Object` (키에 값을 매핑)

조합 계산 (3)

89. 같은 종류끼리 2개를 고르는 조합 수 공식은? $nC2 = n \times (n - 1) / 2$

90. 빈도를 센 뒤 entrySet/Object.entries로 순회하며 조합을 계산하는 흐름을 설명하라

1. 빈도 세기 (패턴 1)로 종류별 개수를 구함
2. entrySet() / Object.entries()로 순회
3. 각 종류의 개수 n에 대해 $nC2$ 계산
4. 모든 종류의 조합 수를 합산

91. 의상 조합 문제에서 "각 종류별 (개수+1)을 곱하고 -1"하는 이유를 설명하라 각 종류에서 "안 입는 경우"를 포함하면 선택지가 (개수 + 1)개. 모든 종류를 곱하면 전체 조합 수. 여기서 "전부 안 입는 경우" 1가지를 빼면 최종 답

Stream / 고차함수 활용 (5)

92. for문 vs Stream/고차함수 중 어떤 기준으로 판단하는가?

- 코테에서는 성능 차이 없음 (수만 건 이하)
- 가독성과 작성 속도 기준으로 선택
- 단순 반복 → for문, 복잡한 체이닝(필터+변환+누적) → Stream/고차함수가 깔끔

93. Java에서 filter → map → reduce 체이닝 코드를 써라

```
java

int result = scores.stream()
    .filter(s -> s >= 60)
    .map(s -> s + 10)
    .reduce(0, Integer::sum);
```

94. JS에서 filter → map → reduce 체이닝 코드를 써라

```
javascript

const result = scores
    .filter(s => s >= 60)
    .map(s => s + 10)
    .reduce((sum, s) => sum + s, 0);
```

95. JS에서 위 체이닝을 변수에 담아 끊어쓰는 코드를 써라

```
javascript
```

```
const filtered = scores.filter(s => s >= 60);
const added = filtered.map(s => s + 10);
const result = added.reduce((sum, s) => sum + s, 0);
```

JS는 각 고차함수가 바로 배열을 반환하므로 끊어쓰기 가능. Java는 매번 collect()로 닫고 stream()을 다시 열어야 해서 체이닝이 자연스러움

96. 중복 제거 + 정렬이 필요할 때 Java와 JS 각각 어떻게 하는가?

```
java

// Java: Stream 체이닝
list.stream().distinct().sorted().collect(Collectors.toList());

// JS: Set + sort (고차함수 없이)
[...new Set(arr)].sort((a, b) => a - b);
```

스택 활용 (2)

97. 연속 중복 제거 문제에서 ArrayList를 스택처럼 쓰는 흐름을 설명하라 배열을 순회하면서, 리스트가 비어있거나 마지막 값과 현재 값이 다르면 add. 같으면 스킵. `isEmpty()` + `get(size()-1)` + `add()` 조합

98. 괄호 유효성 검사에서 스택을 어떻게 활용하는가? 여는 괄호면 push, 닫는 괄호면 pop해서 짝이 맞는지 비교. 순회 끝나고 스택이 비어있으면 유효

```
javascript

const stack = [];
for (const char of s) {
  if ('{[('.includes(char)) {
    stack.push(char);
  } else {
    if (stack.pop() !== pairs[char]) return false;
  }
}
return stack.length === 0;
```

시간복잡도 판단 (7)

99. "두 수의 합이 target인 쌍 찾기" — 어떤 걸 쓰는가? 판단 기준은?

- 데이터 1만 이하 → 이중 반복문 $O(n^2)$ 도 가능
- 데이터 10만 이상 → HashMap $O(n)$ 필수

- HashMap 접근: 순회하면서 `target - current`가 맵에 있는지 $O(1)$ 로 확인

100. 반복문 안에서 존재 여부를 확인해야 할 때 어떤 걸 써야 하는가?

- `ArrayList.contains()` 사용 시: 반복문 $O(n) \times \text{contains } O(n) = O(n^2)$
- `HashSet.contains()` 사용 시: 반복문 $O(n) \times \text{contains } O(1) = O(n)$
- 반복문 안에서 탐색이 필요하다면 HashSet으로 변환한 뒤 사용

101. 정렬 후 탐색 $O(n \log n)$ vs 이중 반복문 $O(n^2)$ — 데이터가 10만 건일 때 어느 쪽? $O(n \log n)$ 선택. 10만 건이면 $n^2 = 100$ 억 연산 $\rightarrow$ 시간 초과. $n \log n \approx 170$ 만 연산 $\rightarrow$ 통과

102. HashMap의 거의 모든 메서드가 $O(1)$ 인 이유를 설명하라 키를 해시 함수에 넣으면 저장 위치(버킷)가 바로 계산되므로, 키 기반의 모든 연산(`put`, `get`, `containsKey`, `remove`)이 $O(1)$

103. `containsValue()`만 $O(n)$ 인 이유는? 값은 해시 함수로 위치를 역추적할 수 없음. 키 $\rightarrow$ 값 방향은 해시로 바로 가지만, 값 $\rightarrow$ 키 방향은 구조가 없어서 전체를 순회해야 함

104. `keySet()`, `values()`, `entrySet()`이 $O(1)$ 인 이유는? 데이터를 복사하지 않고 원본 HashMap을 바라보는 뷰(view) 객체를 반환하기 때문. `subList`와 같은 원리. 뷰를 수정하면 원본도 변경됨

105. 코딩테스트에서 시간 초과가 날 때 가장 먼저 의심해야 할 것은? 반복문 안에서 $O(n)$ 탐색을 하고 있는지 확인. `ArrayList.contains()`, `indexOf()` 등을 반복문 안에서 호출하면 전체 $O(n^2)$ 이 됨. HashSet이나 HashMap으로 바꾸면 $O(n)$ 으로 개선 가능

주의사항 / 실수 방지 (5)

106. Java ArrayList에서 `remove(2)`가 인덱스 제거인지 값 제거인지 어떻게 구분하는가? Java는 int를 우선 매칭하므로 항상 인덱스로 해석. `[1, 2, 3]`에서 `remove(2)` $\rightarrow$ 인덱스 2의 요소(값 3)를 제거 $\rightarrow$ `[1, 2]`. 값 2를 제거하려면 `remove(Integer.valueOf(2))`

107. JS에서 `[10, 9, 1, 2].sort()`의 결과는? 왜 이렇게 되는가? `[1, 10, 2, 9]` — 콜백 없이 `sort()`를 호출하면 모든 요소를 문자열로 변환 후 사전순 정렬. "10"이 "2"보다 앞 (첫 글자 "1" < "2"). 반드시 `sort((a, b) => a - b)` 사용

108. Java subList의 반환값이 뷰라는 건 무슨 의미인가? 복사가 아니라 원본의 일부 구간을 참조하는 객체. 뷰를 통해 값을 바꾸면 원본도 변경되고, 원본을 바꾸면 뷰에도 반영됨. 독립 복사본이 필요하다면 `new ArrayList<> (list.subList(from, to))`

109. JS에서 `splice`와 `slice`를 혼동하면 어떤 문제가 생기는가?

- `splice`: 원본 변경 (제거/삽입)
- `slice`: 원본 보존 (복사본 반환)
- 혼동하면 의도치 않게 원본 배열이 변경되거나, 원본 변경을 기대했는데 안 됨

110. JS 2차원 배열을 `new Array(3).fill(new Array(3).fill(0))`로 만들면 어떤 문제가 생기는가? 모든 행이 같은 배열 객체를 참조. `grid[0][0] = 1`을 하면 모든 행의 [0]이 1이 됨. `Array.from({ length: 3 }, () => new Array(3).fill(0))`으로 만들어야 각 행이 독립

언어 간 대응 (5)

111. Java의 ArrayList에 대응하는 JS 자료구조는? Array. JS의 Array는 Java의 int[] + ArrayList + Stream 역할을 모두 수행

112. Java의 HashMap에 대응하는 JS 자료구조는? 코테에서 주로 쓰는 건? Object 또는 Map. 코테에서는 키가 대부분 문자열이므로 Object를 주로 씀. 키가 문자열 외 타입이면 Map

113. Java의 Comparator에 대응하는 JS 문법은? sort()의 콜백 함수. `(a, b) => a - b`가 Java의 `(a, b) -> a - b`에 대응

114. Java의 Stream에 대응하는 JS 기능은? Array의 고차함수 (filter, map, reduce 등). Java는 `stream()` 진입 + `collect()` 종결이 필요하지만, JS는 배열 메서드를 바로 호출

115. Java에는 있지만 JS에는 없는 메서드, 또는 그 반대를 하나씩 말해라

- Java에만 있는 것: `remove(Object o)` — 값으로 직접 제거. JS에는 없어서 `indexOf` + `splice` 조합 필요
- JS에만 있는 것: `find()`, `some()`, `every()` — 배열에 내장. Java에도 Stream에 `findFirst()`, `anyMatch()`, `allMatch()`가 있지만 코테에서 빈도가 낮음